

ASIST SPRINT 2

System Administration, and Continuous
Integration and Delivery Pipeline for
Business Continuity Project

Prepared by

3DB-G10

João Pinto (1221294)

Alejandro Vieira (1211981)

António Pereira (1220754)

ISEP

Table of Contents

Introduction	03
RFP Module Deployment Automation (UC 6.4.1)	04
Network Access Control (UC 6.4.2)	06
Dynamic Firewall Configuration (UC 6.4.3)	07
Backup Strategy for Minimizing RPO and WRT (UC 6.4.6)	08
Public Folder Access Management (UC 6.4.7)	09
Intrusion Detection for Failed Login Attempts (UC 6.4.8)	10
Conclusion and Future Plans	11



INTRODUCTION

This report outlines the work completed during Sprint 2 of the System Administration (ASIST) Project.

The focus was on implementing automation, enhancing system security, and developing robust user and network management processes.

By leveraging open-source tools like Ansible, Molecule, Fail2Ban, and Podman, we aimed to deliver a **scalable**, **secure**, and **automated infrastructure**.

The following sections detail each objective, the solutions implemented, the challenges faced, and how they were resolved.

RFP Module Deployment Automation (UC 6.4.1)

Objective: Automate the deployment of an RFP module on a DEI VM, ensuring systematic and validated processes using a test plan.

- Ansible was employed for configuration management due to its flexibility and powerful automation capabilities.
- Molecule was used to test Ansible playbooks, simulating deployment environments to validate configurations before production deployment.
- GitHub Actions powered the CI/CD pipeline, automating tasks such as restoring, building, testing, and publishing container images to the GitHub Container Registry (ghcr.io).
- The pipeline ensured every update to the RFP module underwent rigorous testing and was deployed consistently and reliably.

The main goal of the project was to leverage open-source tools to avoid vendor lock-in and maintain maximum flexibility. Ansible was chosen for its robust configuration management capabilities and open-source nature, ensuring that the solution could be adapted or migrated without relying on proprietary software.

GitHub Actions was used for its simplicity and seamless integration with GitHub repositories. However, the pipeline was designed to run standard commands that could just as easily be executed from the command line.

RFP Module Deployment Automation (UC 6.4.1)

```
$ jpnt:[~/Coding/LAPR5/sem5pi-24-25-3db10-back-end/.devops/ansible]> make production
Deploying production environment...
```

```
PLAY [Deploy RFP Backend Module] *****

TASK [rfp_backend : Install Dependencies Task (Debian/apt)] *****
[WARNING]: Platform linux on host vsgate-ssh.dei.isep.ipp.pt is using the discovered Python interpreter at /usr/bin/python3.11; the default Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.17/reference\_appendices/interpreter\_discovery.html for more information.
ok: [vsgate-ssh.dei.isep.ipp.pt]

TASK [rfp_backend : Login into GitHub Container Registry] *****
*****

ok: [vsgate-ssh.dei.isep.ipp.pt]

TASK [rfp_backend : Pull Backend Container Image] *****
*****

ok: [vsgate-ssh.dei.isep.ipp.pt]

TASK [rfp_backend : Stop and Remove Old Backend Container] *****
*****

changed: [vsgate-ssh.dei.isep.ipp.pt]

TASK [rfp_backend : Run Backend Container] *****
*****

changed: [vsgate-ssh.dei.isep.ipp.pt]

PLAY RECAP *****
*****

vsgate-ssh.dei.isep.ipp.pt : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
root@vs:~# podman image ls
REPOSITORY                                TAG      IMAGE ID      CREATED      SIZE
ghcr.io/jpnt/sem5pi-24-25-3db10-back-end  latest   0f04ea53d7fa  27 hours ago  265 MB
root@vs:~# podman container ls
CONTAINER ID   IMAGE                                COMMAND      CREATED      STATUS      PORTS
NAMES
5d702778a22c   ghcr.io/jpnt/sem5pi-24-25-3db10-back-end:latest  /bin/sh     3 seconds ago  Up 3 seconds ago  0.0.0.0:2226->8080/tcp
backend-ctr
root@vs:~# _
```

NETWORK ACCESS CONTROL (UC 6.4.2)

Objective: Restrict system access to clients within DEI's internal network, including cabled and VPN clients.

- Ansible UFW (Uncomplicated Firewall) modules configured firewall rules to restrict access to specific IP ranges.
- Dynamic updates ensured that only internal network clients could connect, enhancing security.

Give your colleagues additional context to your report by using a section header for some introductory message or background. Maximize the next few pages talking about the meat of your report. Make it more detailed and informative by coupling your report's textual information with charts, graphs, and tables, helping you highlight the critical details of your report.

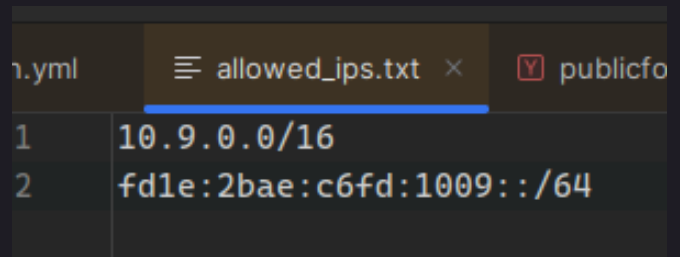
Cap off your presentation with motivational quotes to inspire your colleagues to keep pushing forward for a better and more impressionable report the next time you need to make a new one.

```
---
# SECURE BY DEFAULT: All ports are blocked initially and allow only those specified.
# Resources:
# - https://docs.ansible.com/ansible/latest/collections/community/general/ufw_module.html
- name: Enable UFW Logging
  community.general.ufw:
    logging: 'on'
# NOTE: This breaks idempotence tests (e.g. molecule) but may be useful sometimes to run
# - name: Reset UFW to installation defaults
#   community.general.ufw:
#     state: reset
- name: Set default policy to deny for incoming traffic
  community.general.ufw:
    state: enabled
    default: deny
    direction: incoming
- name: Allow OpenSSH
  community.general.ufw:
    rule: allow
    name: OpenSSH
- name: Allow rate-limited SSH (Prevent Brute-force attacks)
  community.general.ufw:
    rule: limit
    port: ssh
    proto: tcp
- name: Allow multiple ports
  community.general.ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  loop:
    - 2222
    - 2226
- name: Load allowed IPs from file
  set_fact:
    allowed_ips: "{{ lookup('file', 'allowed_ips.txt').splitlines() }}"
```

Dynamic Firewall Configuration (UC 6.4.3)

Objective: Enable updates to the network access list by modifying a simple text file.

- Ansible's lineinfile module was configured to dynamically update firewall rules based on a predefined text file.
- This simplified access list management for system administrators, eliminating manual configuration.



Backup Strategy for Minimizing RPO and WRT (UC 6.4.6)

Objective: Implement a backup strategy to minimize Recovery Point Objective (RPO) and Work Recovery Time (WRT).

- Rsync facilitated incremental backups, copying only changes since the last backup to minimize data loss and backup time.
- Database backups used snapshots stored on a secondary server for consistency and quick restoration.
- Container images were stored in the GitHub Container Registry, ensuring fast restoration of containerized applications.
- Challenge: Preventing interference with system operations during database backups.
- Resolution: Scheduled backups during off-peak hours and utilized database snapshots for consistency.

Public Folder Access Management (UC 6.4.7)

Objective: Create a shared public folder accessible to all registered users for reading files.

- Samba was deployed for shared folder access, integrated with LDAP for user authentication.
- Podman managed containerized environments, mounting shared volumes for scalable storage solutions.
- Storage quotas ensured fair allocation and prevented overuse by individual users.
- Challenge: Setting up proper permissions for the public folder.
- Resolution: Combined Samba and LDAP features to enforce correct permissions, with Podman managing containerized storage.

Intrusion Detection for Failed Login Attempts (UC 6.4.8)

Objective: Detect and mitigate users exceeding 3 failed login attempts.

- Fail2Ban was configured to monitor SSH logs and block IP addresses after three consecutive failed login attempts.
- The SSH daemon was adjusted to log all failed attempts, which Fail2Ban used as triggers for automated action.

```
---
- name: Create jail.local configuration file
  copy:
    dest: /etc/fail2ban/jail.local
    content: |
      [sshd]
      enabled = true
      port = ssh
      filter = sshd
      logpath = /var/log/auth.log
      maxretry = 3
      bantime = 600 # Set to 0 or a small value to avoid banning users
      findtime = 600 # Time window for the 3 failed attempts (10 minutes)
      action = %(action_)s # No blocking action, just logging
      owner: root
      group: root
      mode: '0644'
- name: Start and enable Fail2Ban service
  service:
    name: fail2ban
    state: started
    enabled: true
- name: Restart Fail2Ban service to apply changes
  service:
    name: fail2ban
    state: restarted
- name: Check Fail2Ban status
  command: systemctl status fail2ban # Assuming system with SystemD
  register: fail2ban_status
  ignore_errors: yes
- name: Display Fail2Ban status
  debug:
    msg: "Fail2Ban status: {{ fail2ban_status.stdout }}"
```

Install from the command line

Learn more about packages

```
$ docker pull ghcr.io/jpnt/sem5pi-24-25-3db10-back-end:latest
```

Recent tagged image versions

latest

1.0.0

Published 1 day ago · Digest

sem5pi-24-25-3db10-back-end Container

run on: ubuntu:latest

permissions:

packages: write

contents: read

stages:

- name: Checkout Code

uses: actions/checkout@v4

- name: Setup .NET

uses: actions/setup-dotnet@v4

with:

dotnet-version: 8.0.x

- name: Restore Dependencies

run: dotnet restore sem5pi-24-25-3db10-back-end.sln --runtime linux-x64

- name: Build

run: dotnet build sem5pi-24-25-3db10-back-end.sln --configuration Release --no-restore

- name: Test

run: dotnet test sem5pi-24-25-3db10-back-end.sln --configuration Release --no-build --verbosity normal

- name: Login to GitHub Container Registry

uses: docker/login-action@v2

with:

registry: ghcr.io

username: \${{ secrets.GHCR_USERNAME }}

password: \${{ secrets.GHCR_TOKEN }}

```
reate jail.local configuration file

/etc/fail2ban/jail.local
it: |
id]
led = true
er = sshd
ath = /var/log/auth.log
etry = 3
ime = 600 # Set to 0 or a small value to avoid bannir
time = 600 # Time window for the 3 failed attempts (
ion = %(action_)s # No blocking action, just logging
r: root]
p: root
: '0644'
art and enable Fail2Ban service
:
fail2ban
: started
id: true
start Fail2Ban service to apply changes
:
fail2ban
: restarted
eck Fail2Ban status
: systemctl status fail2ban # Assuming system with Sys
:: fail2ban_status
rrors: yes
isplay Fail2Ban status

'Fail2Ban status: [{ fail2ban_status.stdout }]'
```

```
t:[~/Coding/LAPR5/sem5pi-24-25-3db10-back-end/.devops/ansible]> make production
ing production environment...

[Deploy RFP Backend Module] *****

[rfp_backend : Install Dependencies Task (Debian/apt)] *****
[NG]: Platform linux on host vsgate-ssh.dei.isep.ipp.pt is using the discovered Python interpreter at /usr/
thon interpreter could change the meaning of that path. See
://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html for more information.
vsgate-ssh.dei.isep.ipp.pt]

[rfp_backend : Login into GitHub Container Registry] *****
*****
vsgate-ssh.dei.isep.ipp.pt]

[rfp_backend : Pull Backend Container Image] *****
*****
vsgate-ssh.dei.isep.ipp.pt]

[rfp_backend : Stop and Remove Old Backend Container] *****
*****
d: [vsgate-ssh.dei.isep.ipp.pt]

[rfp_backend : Run Backend Container] *****
*****
d: [vsgate-ssh.dei.isep.ipp.pt]

RECAP *****
*****
>-ssh.dei.isep.ipp.pt : ok=5 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored
```

Conclusion

Sprint 2 delivered a robust and secure infrastructure, achieving all key objectives. The successful deployment of automation pipelines, firewall configurations, backup strategies, and user management systems reflects careful planning and effective problem-solving.